

MODE COLLAPSE UNDER GRPO: MODEBENCH AND ONLINE VERIFIED MAXIMUM ENTROPY

Anonymous authors

Paper under double-blind review

ABSTRACT

Repeated sampling helps only when a policy proposes distinct viable solutions. Binary-reward RL can raise accuracy while concentrating on one correct execution, so eight attempts buy one answer. We introduce **MODEBENCH**, five executable domains whose validators jointly determine correctness and outcome identity, and **xGRPO**, which rewards and replays rare verified outcomes. On a preregistered five-seed, 12-epoch surface covering all five domains on one common design, xGRPO raises `pass@8` and `distinct@8` everywhere. **PantryPlan** exposes the trajectory: **Dr.GRPO** raises `pass@1` .388 $\rightarrow$.525 while `pass@8` falls .902 $\rightarrow$.556 and `distinct@8` 2.426 $\rightarrow$.634; xGRPO ends at .869 and 2.186 with a *higher* `pass@1` of .605. Analysis explains balanced replay, and a six-run MathIR ablation isolates +.137 `distinct@8` from separated support discovery.

1 INTRODUCTION

Test-time scaling promises to trade inference compute for better reasoning by drawing multiple candidates and selecting among them. We study a failure that breaks that trade: binary-verifier RL can raise accuracy while concentrating a policy on one successful execution. Under group-relative binary-reward RL, eight nominal attempts can become eight copies of the same answer—the system looks better by correctness while its useful proposal support quietly collapses.

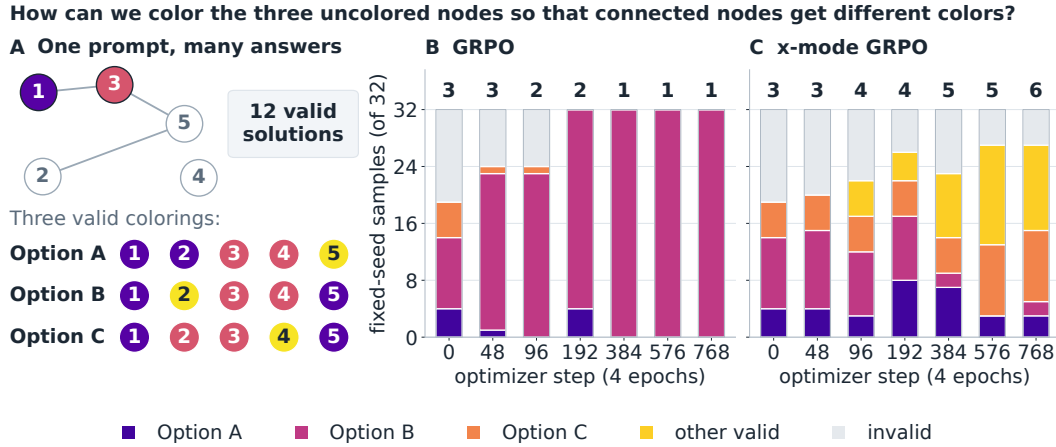


Figure 1: Mode-collapse example. (A) One prompt has 12 valid colorings; three are shown. (B–C) Four fixed-seed $K = 8$ draws from matched Qwen2.5-0.5B-Instruct trajectories; color denotes an executed correct mode and gray an invalid response. Dr.GRPO reaches 32/32 correct but contracts to one mode; xGRPO retains six. Accuracy rises while the marginal value of sampling collapses.

Breadth is what test-time scaling consumes. Self-consistency [33], best-of- N [4], verifier-guided search [29], and self-improvement [8] all spend candidates, and a selector returns only a route the

policy proposes. Breadth therefore decides whether more inference buys another chance or another copy: RLVR can win at small k yet lose at large k [36], while RLHF and fine-tuning can trade generalization or `pass@K` for narrower outputs [16, 7]. Beyond exact tasks, a one-bit reward flattens what it means to be right because a distribution over acceptable answers and a point mass earn the same score. Exact domains make this loss countable; plurality matters everywhere. Standard alignment can reduce distributional pluralism [31]; related studies report a “correct-answer effect” on nuanced questions and flattened portrayals of demographic groups [25, 32].

Useful breadth is difficult to define and preserve. String diversity overcounts aliases, so answers need meaning-level clustering [10], and final answers undercount the programs that reach them [33]. Valid modes are unknown, credit must reach verified novelty alone where novelty search and count-based bonuses pay for any unvisited behavior [17, 2], and rare modes can vanish before a prompt returns [26]. Binary reward equates all correct responses [27, 23], while finite on-policy sampling reinforces common modes and leaves absent modes untouched [6]. Repairing the failure therefore takes a signal bound to executed mode identity.

Prior work documents collapse and promotes outcome bonuses, batch penalties, `pass@K` objectives, and set-level or distributional optimization [6, 7, 30, 34, 5, 20]. But an open-set executable task needs three properties jointly: semantic identity bound to correctness, credit only for a genuinely new verified outcome, and memory that preserves a transient discovery beyond its collection batch. Each supplies one or two; closing the gap takes all three.

We introduce MODEBENCH, whose five domains derive correctness and canonical mode identity from the same execution, and xGRPO, a Dr.GRPO extension over the policy’s own verified outcomes. xGRPO rewards rare and newly discovered verified outcomes, retains them in a prompt-local bank, and schedules global verified replay with mass and balance control. On the common five-seed surface, matched Dr.GRPO makes neutral `pass@1`, neutral `pass@8`, and mean # distinct correct@8 nearly coincide; xGRPO opens that band in every domain. PantryPlan makes the loss temporal: Dr.GRPO raises `pass@1` .388 $\rightarrow$.525 while `pass@8` falls .902 $\rightarrow$.556 and `distinct@8` 2.426 $\rightarrow$.634; xGRPO retains far more sampled support at strictly higher accuracy. A completed MathIR ablation isolates +.137 `distinct@8` from separated support discovery. The exact 0.5B environments are synthetic, and the MathIR actuator comparison supplies the component-level causal claim.

Summary of contributions.

- *Problem and measurement* (Sections 2–3): we separate correctness from executed correct-mode support and release MODEBENCH, five domains whose validators bind reward and canonical identity.
- *Mechanism and analysis* (Section 4 and Appendix A): xGRPO combines open-set discovery credit, rare-mode reward, and global verified replay; a categorical analysis shows when retained-support balance rules out winner-take-all collapse.
- *Audited evidence* (Sections 6–7): preregistered five-seed trajectories on a fixed checkpoint grid establish one consistent direction across all five completed domains; PantryPlan records accuracy rising as sampled support collapses, while a six-run MathIR ablation causally isolates the separated support actuator. Limitations are explicit in Section 9.

2 MODE COLLAPSE UNDER GRPO

The unit of diversity must be semantic. Surface strings overcount formatting changes, while final answers can erase different programs or procedures. We therefore identify a mode with a validator-positive executed outcome: the same execution determines whether a response is correct and which canonical outcome it realizes. Section 3 gives the exact benchmark contract; this section formalizes the failure it exposes.

2.1 CORRECT MASS AND CORRECT-MODE DISTRIBUTION

For prompt x , response y , and reference specification s_x , let

$$V(y, s_x) \in \{\perp\} \cup \mathcal{C}_x, \quad r(x, y) = \mathbf{1}[V(y, s_x) \neq \perp].$$

The policy’s total correct mass is $P_\theta^+(x) = \Pr_{Y \sim \pi_\theta}[V(Y, s_x) \neq \perp]$. Conditional on correctness, its mode distribution is

$$p_\theta^+(c | x) = \Pr[V(Y, s_x) = c | V(Y, s_x) \neq \perp].$$

We say training exhibits *mode collapse* on x when probability within p_θ^+ concentrates on a strict subset of $\mathcal{C}_x$ while P_θ^+ remains comparable or increases. Mode collapse is therefore a statement about where correct mass sits: a high-accuracy policy on one correct mode exhibits it.

Two population quantities describe the failure: conditional verified-mode entropy $H_\theta^+(x) = -\sum_c p_\theta^+(c | x) \log p_\theta^+(c | x)$ and concentration $\max_c p_\theta^+(c | x)$. They are unavailable when the support is unknown. Given K independent samples $Y_{1:K}$, define

$$D_K(x) = |\{V(Y_i, s_x) : V(Y_i, s_x) \neq \perp\}|, \quad P_K(x) = \mathbf{1}[D_K(x) > 0].$$

Evaluation reports $\text{distinct@}K = \mathbb{E}_x \mathbb{E}[D_K(x)]$ and $\text{pass@}K = \mathbb{E}_x \mathbb{E}[P_K(x)]$. The first counts correct execution modes; the second asks whether search finds any. We also report $\text{mean@}K$, the fraction of draws that are correct, and greedy $\text{pass@}1$. Each carries information the others leave out.

Two structural properties make this quartet a single diagnostic. Because $D_K \mathbf{1}[D_K > 0] = D_K$, their ratio is an exact conditional expectation, $\text{modes-per-success} = \text{distinct@}K / \text{pass@}K = \mathbb{E}[D_K | D_K > 0]$, the number of *different* correct executions a successful K -sample set contains; a value of 1 means every success is the same success. And $\text{mean@}K \leq \text{pass@}K \leq \text{distinct@}K$ always, with the first inequality strict unless temperature-one sampling is degenerate. A tight band across all four numbers is therefore a signature of a point-mass policy.

2.2 WHY EQUAL REWARD CONCENTRATES CORRECT MODES

For a group of G responses, both variants center the binary reward on its group mean, $A_i^{\text{task}} = r_i - G^{-1} \sum_{j=1}^G r_j$; Dr.GRPO applies one shared maximum-length normalization where GRPO divides by the group reward standard deviation. Each rescaling is blind to *which* correct mode a response realizes, so correct responses in the same group receive the same task advantage. That symmetry equalizes credit across identities and leaves mode ratios free to follow sampling frequency; Lemma A.1 shows the two normalizations share a direction and differ only in speed.

A categorical toy model makes the feedback explicit. Let z_c be a logit for correct mode c , $p_c = \text{softmax}(z)_c$, and $P^+ = \sum_{c \in \mathcal{C}} p_c$. Gradient ascent on expected binary reward gives

$$\dot{z}_c = \frac{\partial P^+}{\partial z_c} = p_c(1 - P^+), \quad \frac{d}{dt} \log \frac{p_c}{p_d} = (p_c - p_d)(1 - P^+).$$

While incorrect mass remains, the more common of two equally rewarded correct modes receives the larger logit update. Finite on-policy groups sharpen the asymmetry: an absent rare mode receives no update, and an all-correct group has zero centered task advantage. Thus the first successful mode can become a self-reinforcing and eventually locally frozen solution. This idealized calculation isolates the mechanism: under binary reward the only force acting on mode ratios is sampling frequency. Recent analyses reach related conclusions through outcome-frequency amplification, divergence choice, or set-level policy geometry [28, 21, 19].

Appendix A makes this precise. Under the infinitesimal on-policy mean dynamics, Theorem A.2 shows that generic binary-reward GRPO and Dr.GRPO trajectories select one correct mode, and Theorem A.5 with Corollary A.6 shows that once the verified bank covers the support, xGRPO reverses the unstable mode-ratio dynamics. The coverage assumption is stated explicitly: the theorems act once the bank covers the support.

3 MODEBENCH: EXECUTABLE MODE MEASUREMENT

3.1 BENCHMARK CONTRACT

A benchmark instance is mode-verifiable only when V satisfies two requirements. First, it executes the object the model actually generated and judges that execution. Second, the canonical key is derived from the same execution that established correctness. Cosmetic aliases merge; different

Domain	Generated answer / check	Executed key	Aliases that merge	mean valid modes per eval prompt	train/eval
Graph coloring	Complete assignment; check fixed colors and edges	Color vector	Spacing only	6.27	384/128
Countdown	Parse operands; execute exactly	Normalized AST	Commutative order	4.52	384/128
Python factors	Execute restricted lambda externally	Return vector	Source programs with same behavior	229.44	384/128
MathIR	Execute bounded action program	State trajectory	Programs with the same states	5.00	384/128
PantryPlan	Exact ingredient allocation; check all bounds	Ingredient support	Quantity and ordering aliases	18.19	384/128

Table 1: **The MODEBENCH contract.** Mean valid modes are exact over the released multi-answer evaluation split. Correctness and identity come from one executable path; training starts without the mode catalogue. The exact prompt each policy receives, per domain, is reproduced verbatim in Appendix C.

executed outcomes stay distinct. This contract confines diversity credit to verified responses and confines breadth to solutions the validator has distinguished.

Graph coloring. Each prompt gives a graph, a partial three-coloring, and hidden vertices; the validator reconstructs the completion, preserves shown colors, and rejects monochromatic edges. The color vector is the key.

Countdown. Each prompt gives three distinct integers and a target, each usable once with $+$, $-$, $\times$, $\div$ and parentheses. The parser executes in exact rational arithmetic and canonicalization sorts commutative children, merging syntactic aliases while retaining genuinely different trees.

Python factors. Each prompt asks for `lambda n: EXPR` on four composite inputs; a syntax-restricted external worker (Appendix B) requires every returned d to satisfy $1 < d < n$ and $n \bmod d = 0$. The output vector is the key, so different program texts with identical behavior merge. A prompt with inputs $(n_1, \dots, n_m)$ has exactly $\prod_j |D(n_j)|$ modes for $D(n)$ the proper nontrivial divisors; released rows contain 16–3,600.

MathIR. Each prompt shows a linear equation and at most eight lettered transformations, of which the model emits at most four. An interpreter applies each to both sides and normalizes exactly after every step; the key is the state trajectory. Each prompt has five exhaustively enumerated valid routes, withheld from the policy and the bank.

PantryPlan. Each prompt asks for an exact integer-gram allocation from six frozen ingredients subject to inventory, serving-step, mass, nutrient, and dietary-tag constraints. The same decimal-arithmetic validator accepts the allocation and keys it by the sorted set of ingredients used: quantity and ordering aliases merge, while different feasible formulations remain distinct. The registered evaluation split averages 18.19 exact supports per prompt, all withheld from the policy and bank.

Prompts. Every domain is a single-turn prompt with a fixed chat template and no few-shot examples; the exact rendered string the policy receives in each domain, taken from the first row of that cohort’s frozen evaluation split, is reproduced verbatim in Appendix C.

4 XGRPO: ONLINE VERIFIED MAXENT-DR.GRPO

xGRPO—short for *executable-mode GRPO*—is an outcome-level maximum-entropy surrogate layered on Dr.GRPO: it spreads mass over executed verified outcomes and learns their support online, coupling rare/new-mode advantages, verified replay, and self-referenced adaptive coefficients while leaving the task advantage unchanged. One design rule governs every component: *a signal may act on an outcome only after that outcome has been executed and accepted*, so discovery credit, replay, and balance all read the same validator output.

One prompt, two verified answers, two different keys			
	response	execute + verify	canonical key
A Graph coloring	partial colors 2??1?2 · fill vertices 2, 3, 5		
■ ANSWER 1	113	✓ valid coloring	211132
■ ANSWER 2	131	✓ valid coloring	213112
B Countdown	tiles {3, 6, 9} · target 18 · use each tile once		
■ ANSWER 1	(6 × 9) / 3	✓ = 18	div(mul(6,9),3)
■ ANSWER 2	6 + 3 + 9	✓ = 18	add(9,add(3,6))
C Python factors	lambda n: EXPR · called once per n in 18, 82, 91, 93 · return a proper factor d		
■ ANSWER 1	lambda n: 2 if n%2==0 else (7 if n%7==0 else 3)	✓ divides each n	[2, 2, 7, 3]
■ ANSWER 2	lambda n: 3 if n%3==0 else (13 if n%13==0 else 41)	✓ divides each n	[3, 41, 13, 3]
D MathIR	x/2 - 9 = -1 · C: subtract -9 · F: x2 · E: subtract -18		
■ ANSWER 1	C;F	✓ x = 16	x/2 = 8 → x = 16
■ ANSWER 2	F;E	✓ x = 16	x - 18 = -2 → x = 16
E PantryPlan	2-4 ingredients · 125-200 g · four exact nutrition bounds		
■ ANSWER 1	navel_orange=75; sunflower_seeds=50	✓ feasible	navel_orange + sunflower_seeds
■ ANSWER 2	grape_tomatoes=75; almonds=50	✓ feasible	almonds + grape_tomatoes

Figure 2: **Two verified modes for one prompt in each MODEBENCH domain.** Every row reads left to right in the order the pipeline runs: the response the policy emits, the execution that accepts it, and the canonical key that same execution produces. Graph answers complete 2??1?2; Countdown keys are normalized ASTs; the Python lambda is called once per input, so its key is the four returned factors in input order; MathIR keys are state trajectories; and PantryPlan keys are ingredient supports. In every domain both answers are correct, but their keys differ. The two answer colours are the two verified-mode colours of Figure 1. Every domain uses the first row of its fixed evaluation split. The full prompt behind every domain is in Appendix C.

Each component below was added in response to a prospectively recorded failure of the preceding version; Appendix H gives that record and what each component’s evidence licenses, and Algorithm 1 in Appendix E fixes the exact update.

4.1 ONLINE VERIFIED SUPPORT

For each prompt x the learner maintains a bank $\mathcal{B}_x^+$, initially empty, of keys the current policy generated and V accepted. Every group is scored against one immutable pre-group snapshot and new keys are committed only after the whole group, so novelty is invariant to row order. Admission is fail-closed at the intersection of positive rollout reward and executable canonical validation; a disagreement receives no key and is recorded. The matched Dr.GRPO control runs the same validator and the same passive counts with all auxiliary coefficients set to zero, so discovery telemetry is comparable across arms without altering the control’s objective.

4.2 OPEN-SET RARE-MODE AND DISCOVERY ADVANTAGES

For a verified mode a , let $c_x(a)$ combine its historical count with its leave-one-out multiplicity among the other verified responses in the group. With pseudocount $\lambda = 1$, the detached predictive distribution $\hat{q}_x$ over the J_x explicitly counted keys includes one structural unseen bucket $\emptyset$:

$$\hat{q}_x(a) = \frac{c_x(a) + \lambda}{\sum_b c_x(b) + \lambda(J_x + 1)}, \quad \hat{q}_x(\emptyset) = \frac{\lambda}{\sum_b c_x(b) + \lambda(J_x + 1)}.$$

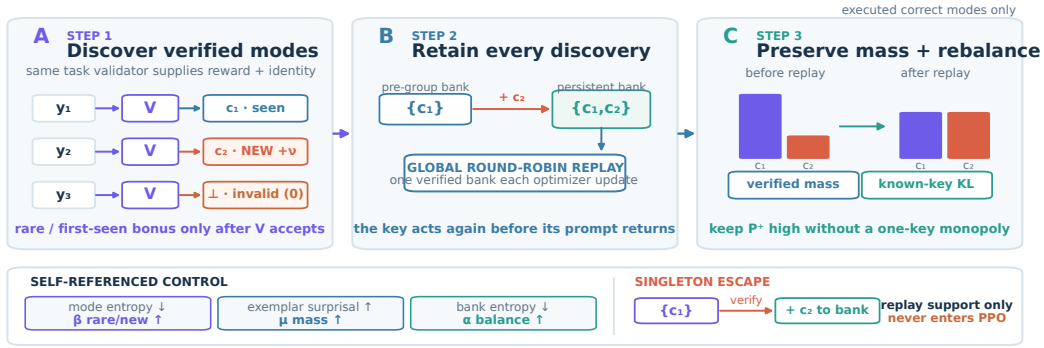


Figure 3: **xGRPO as one discover–retain–rebalance loop.** (A) The task validator rejects invalid novelty and rewards only rare or new correct keys. (B) New keys enter a persistent bank whose global schedule replays them after the collection batch. (C) Replay protects total verified mass while balancing probability across known keys. The lower controller raises the matching pressure when its sensor deteriorates; a singleton-only escape may add a verified alternative to replay support, never to PPO.

The unseen bucket reserves mass for modes the bank has yet to see. This is an executable-outcome analogue of count-based exploration [2]: only validator-positive visits receive a count or bonus.

With surprisal clip $S = 5$, define $h_i = \min\{-\log \hat{q}_x(a_i), S\}$ and $\bar{h} = \mathbb{E}_{a \sim \hat{q}_x} \min\{-\log \hat{q}_x(a), S\}$, and set $A_i^{\text{rare}} = \beta_t (h_i - \bar{h}) / S$. Centering under $\hat{q}_x$, rather than under the empirical group, keeps a carrier when the current correct samples all share one mode but the bank records more. Let $\mathcal{D}_x$ be the verified keys absent from the pre-group bank and $m_x(a)$ their current-group multiplicity. A first discovery receives $A_i^{\text{new}} = \nu \mathbf{1}[a_i \in \mathcal{D}_x] / m_x(a_i)$ with $\nu = 0.50$, so all duplicates of one new key share total credit ν . The actor advantage is

$$A_i = A_i^{\text{task}} + \mathbf{1}[V(y_i, s_x) \neq \perp] (A_i^{\text{rare}} + A_i^{\text{new}}).$$

The mode terms enter uncentered, and novelty and entropy pressure reach validator-positive responses alone.

4.3 GLOBAL VERIFIED REPLAY

With hundreds of training prompts, a prompt-local discovery would otherwise wait nearly a pass before acting again. After the first verified bank exists, every optimizer update therefore schedules exactly one bank in deterministic global round-robin order, retaining at most one policy-generated exemplar per observed key up to 16 keys. Replay converts a transient discovery into a persistent training constraint, paralleling rehearsal’s role in resisting forgetting [26]. Two losses apply: *verified mass*, uniform exemplar likelihood for every scheduled bank including a singleton, with coefficient μ_t ; and *known-mode balance*, $\text{KL}(U_{\text{bank}} \| q_{\text{model}})$ for banks with at least two observed modes, with coefficient α_t . Selection reads only accumulated verified banks and a one-bank compute budget—never exhaustive support, evaluation outcomes, a desired mode count, or reference solutions. Before the first discovery, replay is inactive.

4.4 PROJECTION-FREE SELF-REFERENCE

The rare-mode, verified-mass, and balance coefficients each warm up for 64 eligible observations and reference only their own warmup mean. Let ρ be normalized open-set entropy, s verified-exemplar surprisal, and h scheduled bank entropy. With EMA decay .9,

$$\beta_t = .10 \frac{\rho_{\text{ref}}}{\rho_{\text{ema}}}, \quad \mu_t = .10 \frac{s_{\text{ema}}}{s_{\text{ref}}}, \quad \alpha_t = .10 \frac{h_{\text{ref}}}{h_{\text{ema}}}.$$

Falling mode entropy increases exploration pressure; rising surprisal on known correct exemplars increases mass pressure; falling bank entropy increases balance pressure. No coefficient is projected. This is a stress-test design: continued collapse remains visible as continued coefficient growth rather than as silent saturation. Singleton banks are omitted from the balance sensor.

4.5 SEPARATED-SUPPORT ESCAPE ACTUATOR

All balance mechanisms need an alternative to preserve, and a singleton bank offers none. A registered extension therefore tests one support-only escape, eligible only after controller warmup and only when the run’s own entropy EMA falls below its own warmup reference: a domain-specific, validator-preserving transformation may propose at most one new verified key. Proposal rows never enter PPO, and proposal-only support is isolated from the historical count table behind the on-policy rare/new advantages—it may enter replay support, but joins on-policy counts only once the neutral policy independently produces and validates it. A same-plumbing actuator-off arm is the causal comparator.

5 RELATED WORK

Inference-time compute and verifier-based reasoning RL. Self-consistency, repeated sampling, and verifier-guided search generate a candidate set and then aggregate or select from it [33, 4, 29, 22]. GRPO and Dr.GRPO instead train proposal policies from group-relative outcome rewards [27, 12, 23]. Dr.GRPO fixes response-length normalization; our upstream question is orthogonal: whether RL preserves enough probability *within* equally rewarded correct outcomes for additional inference to remain useful.

Collapse and diversity-aware optimization. RLHF trades generalization against diversity [16], RLVR can lose token entropy [6] and can narrow the large- k sampling boundary relative to the base model [36], and fine-tuning can raise `pass@1` while `pass@K` falls [7]. Outcome bonuses, batch penalties, `pass@K` objectives, semantic similarity, mass-covering divergences, set objectives, and distribution matching target related failures [30, 34, 5, 20, 19, 21, 28]; clipping asymmetries such as Yu et al. [35] counteract entropy collapse at the token level. Closest is Song et al. [30]: their UCB counts final answers, including incorrect ones, their batch method penalizes repeats, and bonuses are disabled on all-correct batches; their 7B competition-math study is stronger external evidence for the general problem. xGRPO addresses a narrower distinction: only validator-positive *executed* keys receive novelty, and replay lets a verified discovery act beyond its collection batch.

Mode-covering objectives and archives. GFlowNets sample in proportion to reward rather than maximizing it, and have been used to fine-tune language models toward diverse high-reward reasoning chains [3, 15, 37]. Quality-diversity search maintains an explicit archive indexed by a behavior descriptor and selects for novelty as well as fitness [17, 24]. xGRPO’s bank is a quality-diversity archive whose behavior descriptor is the validator’s own canonical key, and its balance loss targets the uniform-over-verified-support distribution rather than the reward-proportional one. Execution supplies identity, so the validator’s key replaces the hand-designed descriptor space or tractable target density both literatures require.

Semantic maximum-entropy control. Maximum-entropy RL augments reward with policy entropy and can adapt its coefficient [38, 18, 13, 14]; grammar constraints can prevent invalid generations [11]. xGRPO changes the random variable from tokens or a known action set to open-set verified executions, combining empirical rare-mode advantages with mass/balance replay into an outcome-level surrogate estimated from the bank. Where semantic entropy clusters generations by bidirectional entailment to estimate uncertainty [10], MODEBENCH makes the equivalence relation exact and adversarially safe by construction: Python uses a restricted grammar before isolated execution and MathIR a finite command language, so the key cannot be asserted by the model. The contract removes many false distinctions without claiming every reasoning strategy is executable.

6 EXPERIMENTAL DESIGN AND EVIDENCE POLICY

6.1 MATCHED TRAINING AND EVALUATION

All five domains run one preregistered design: Qwen2.5-0.5B-Instruct at a pinned revision, seeds 43–47, group size 16, $\beta_{\text{KL}} = 0$, rollout temperature 1, and exactly 12 epochs through each fixed pool. Every domain trains on 384 prompts and evaluates on a fixed 128-prompt split, giving 4,608 optimizer updates per run; earlier smaller-pool cohorts are superseded by that common size and are not reported. PantryPlan’s task-bound interface selects a six-ingredient binary support mask and materializes one feasible allocation, so it tests support retention rather than free-form recipe

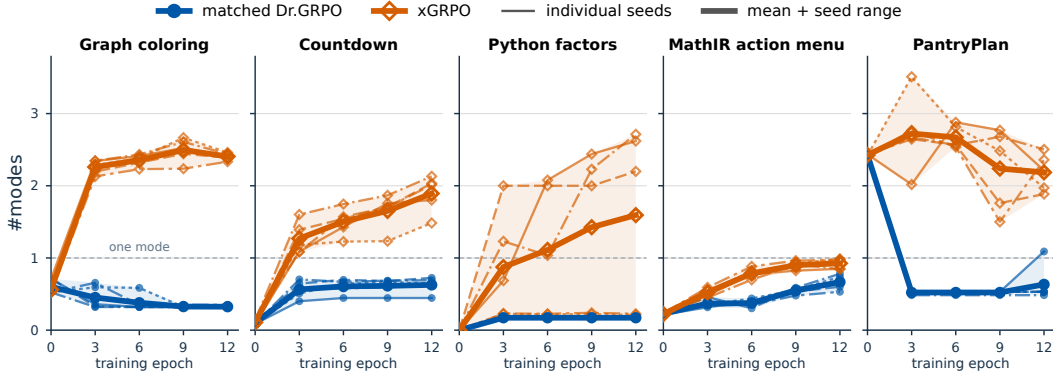


Figure 4: **Average number of verified modes (#modes) across training**, one panel per MODEBENCH domain on a shared scale; thin lines are seeds 43–47, thick lines five-seed means, bands seed ranges. Below the dashed rule a second sample cannot return a second execution. Endpoints reproduce Table 2.

generation. Within every domain, arms share data, model, optimizer, rollout count, training budget, evaluation seed, checkpoint cadence, and recovery rules; only the xGRPO mechanism differs.

Evaluation uses greedy decoding and four deterministic replicates of $K = 8$ temperature-one samples, run four times per training pass. We report the five fixed checkpoints 0, 3, 6, 9, 12; every headline row is the deepest checkpoint reached by all five seeds of both arms, which is the terminal pass 12 in every domain.

Is the gain bought with extra compute? The arms take the same optimizer steps over the same rollouts, so both perform identical *generation*—the dominant cost in verifier RL. The extra work is scoring at most 16 already-generated exemplars per update at weight $w_{\text{rep}} = (G - 1)/G^2 \approx .06$ against a 16-rollout group, every one of them produced and verified by the policy itself earlier in the same run: a bounded reuse term over the same sample budget.

6.2 EVIDENCE POLICY

All five domains are complete and terminally audited on this one design: the same seeds, arms, reporting rule, and pass-12 endpoint, with each domain standing on its own rather than pooled into a single test. The MathIR same-plumbing actuator comparison supplies the component-level causal claim. The evidence ledger, state-integrity rules, exclusions, dataset auditing, seed rows, terminal causal rows, and cohort provenance are bound in Appendices K, D, F, G, and J.

7 RESULTS

7.1 MATCHED DR.GRPO COLLAPSES THE WHOLE METRIC BAND

At the reported endpoints, matched Dr.GRPO’s four metrics coincide: the spread $\text{distinct@8} - \text{mean@8}$ never exceeds .109 and modes-per-success never exceeds 1.14 (Table 2), so a successful eight-sample set rarely holds a second execution. Temperature-one mean@8 tracks greedy pass@1 to within .002: the band measures a point-mass policy, and no temperature reopens it (Appendix I).

The failure sits in support while accuracy holds. Graph holds greedy accuracy at .323 and Countdown rises to .588 while modes-per-success stays pinned at 1.00 and 1.06, against 6.27 and 4.52 valid modes per evaluation prompt (Table 1). On Python, where the mean prompt admits 229 valid programs, all five baseline seeds land on .172 for all four metrics: one deterministic program.

7.2 xGRPO REOPENS THE BAND; PANTRYPLAN EXPOSES THE COLLAPSE

Across the five domains, xGRPO raises pass@8 in all twenty-five paired seed cells and greedy pass@1 by +.033 to +.511. Graph and Countdown mainly gain breadth (modes-per-success

Domain	epoch	matched Dr.GRPO				xGRPO			
		p@1	m@8	pass@8	#modes	p@1	m@8	pass@8	#modes
Graph coloring	12	.323	.322	.325	.325	.463	.407	.961	2.406
Countdown	12	.588	.587	.594	.627	.620	.610	.693	1.893
Python factors	12	.172	.172	.172	.172	.683	.674	.688	1.594
MathIR action menu	12	.645	.644	.666	.666	.766	.735	.867	.926
PantryPlan	12	.525	.524	.556	.634	.605	.566	.869	2.186
<i>band spread (#modes−m@8)</i>		.000–.109				.190–1.999			
<i>modes-per-success</i>		1.00–1.14				1.07–2.73			

Table 2: **Five-domain endpoint means on one common design.** Every cell is a five-seed (43–47) mean of both arms at the common terminal epoch 12. **#modes** is `distinct@8`: the average number of distinct verified modes an eight-sample draw returns, which is what repeated sampling actually buys. xGRPO reopens sampled support in all five domains and raises `pass@8` in all 25 paired seed cells. Seed rows are in Appendix F.

1.00 → 2.50 and 1.06 → 2.73), while Python and MathIR mainly gain a first route: MathIR moves 1.00 → 1.07 but its `pass@8` rises .666 → .867. Python remains seed-unstable: three xGRPO seeds reach `pass@8` 1.000, two land at .211 and .227. Figure 4 shows the same comparison as a trajectory: the baseline holds below one mode throughout, xGRPO separates by epoch 3.

PantryPlan makes the failure temporal. Dr.GRPO raises `pass@1` .388 → .525 and `mean@8` .344 → .524, yet collapses `pass@8` .902 → .556 and `distinct@8` 2.426 → .634, almost all of it inside the first three passes, after which its four metrics are numerically identical at .522 through pass 9. xGRPO instead *raises* support early and ends at .605, .566, .869, and 2.186, ahead on all four metrics: accuracy improves for the baseline while “right” is flattened from several feasible plans to exactly one.

7.3 TERMINAL CAUSAL RESULT: SEPARATED SUPPORT DISCOVERY

The completed MathIR same-plumbing ablation changes only separated replay support. At pass 12 it raises `pass@1` .7708 → .7969, `pass@8` .8555 → .8822, and `distinct@8` .9102 → 1.0469 in all three pairs; 327 interventions enter neither PPO nor on-policy counts (Appendix G).

8 DISCUSSION

Three distinctions organize the design. *Identity*: accepted execution defines a mode. *Discovery versus balance*: balance acts on modes already banked, so the isolated actuator targets coverage. *Entropy versus support*: token entropy raises per-position uncertainty; support counts verified keys. The next test is a coverage–budget curve on larger executable tasks.

9 LIMITATIONS

The common five-seed design and fixed reporting grid remove available-seed averaging, but each domain still stands on its own rather than in a pooled test. All domains are synthetic, the model is 0.5B, and exact identity is unusually available. PantryPlan tests substitution within a declared item set and Python Factors a restricted grammar in isolation, so our claims stop at those interfaces. Baselines are limited to matched Dr.GRPO; beyond decoding (Appendix I), matched token entropy and distribution matching remain untested, as does whether a selector turns breadth into end-task accuracy.

Conclusion. Group-relative binary reward can improve accuracy while flattening “right” to one answer. MODEBENCH makes that lost option value measurable across five executable domains, categorical analysis explains it, and xGRPO implements a discover–retain–rebalance response. The completed five-seed surface shows the same support-opening direction in every domain, and the MathIR ablation isolates separated discovery. Exact verification is our microscope: elsewhere many answers satisfy the same goal, and preserving one narrows what selection recovers.

ETHICS STATEMENT

The verified world is a microscope: what is mathematical here is our ability to name and validate modes. Preserving the diversity of what it means to be right is therefore option value for reasoning, selection, and collective expression. Creative-work studies find that AI assistance can raise average ratings while making stories more similar and users less semantically distinct [9, 1]. These studies show that quality and a narrower collective space can coexist; establishing Dr.GRPO as the cause and showing that xGRPO transfers to open-ended tasks both remain open. Where modes are hardest to enumerate, they may be easiest to erase.

This cuts both ways in deployment. Breadth is instrumentally valuable: a policy proposing more distinct correct executions may also propose riskier ones, and `distinct@K` credits an alternative without asking what acting on it costs. Systems adopting this objective should report correctness, invalidity, and breadth together, and should map executed modes onto application-valued distinctions rather than treating mode count as an end in itself. All experiments here use synthetic, self-generated data with no human subjects and no personal or scraped content.

REPRODUCIBILITY STATEMENT

Every quantitative claim is bound to a frozen artifact. One matching caveat is explicit: GPU placement is not matched across arms, and it is not inert, because the two GPU models in this cohort sample different tokens from identical weights under identical draw seeds. Twenty-two of the twenty-five paired seeds ran both arms on one GPU model and three pairs are split; the decoding control of Appendix I pins every measurement to the GPU model that trained its checkpoint. Section 6 and Appendix K give the matched protocol, the complete hyperparameter set, and the evidence ledger that fixes what each comparison may claim; Appendix D records SHA-256 identities for all training and evaluation splits. Comparison protocols were registered before results were observed and are included as `paper/preregistration/*.md`. Bank, controller, scheduler, optimizer, data-cursor, and request-stream state are checkpointed atomically, and integrity violations—non-finite values, state mismatch, missing terminal markers, or proposal leakage into PPO—fail closed rather than degrade silently. Every figure is regenerated from hash-bound artifacts by a named script listed in Appendix K, and rendering aborts if a displayed endpoint does not reproduce its frozen aggregate. Two superseded cohorts are excluded by machine-readable audits written before their outcomes were known.

REFERENCES

- [1] Barrett R. Anderson, Jash Hemant Shah, and Max Kreminski. Homogenization effects of large language models on human creative ideation. In *Proceedings of the 16th Conference on Creativity & Cognition*, pp. 413–425. Association for Computing Machinery, 2024. doi: 10.1145/3635636.3656204.
- [2] Marc G. Bellemare, Sriram Srinivasan, Georg Ostrovski, Tom Schaul, David Saxton, and Remi Munos. Unifying count-based exploration and intrinsic motivation. In *Advances in Neural Information Processing Systems*, volume 29, 2016. URL <https://arxiv.org/abs/1606.01868>.
- [3] Yoshua Bengio, Salem Lahlou, Tristan Deleu, Edward J. Hu, Mo Tiwari, and Emmanuel Bengio. GFlowNet foundations. *Journal of Machine Learning Research*, 24(210):1–55, 2023.
- [4] Bradley Brown, Jordan Juravsky, Ryan Ehrlich, Ronald Clark, Quoc V. Le, Christopher Ré, and Azalia Mirhoseini. Large language monkeys: Scaling inference compute with repeated sampling, 2024. URL <https://arxiv.org/abs/2407.21787>.
- [5] Zhipeng Chen, Xiaobo Qin, Youbin Wu, Yue Ling, Qinghao Ye, Wayne Xin Zhao, and Guang Shi. Pass@k training for adaptively balancing exploration and exploitation of large reasoning models, 2025. URL <https://arxiv.org/abs/2508.10751>.
- [6] Ganqu Cui, Yuchen Zhang, Jiacheng Chen, Lifan Yuan, Zhi Wang, Yuxin Zuo, Haozhan Li, Yuchen Fan, Huayu Chen, Weize Chen, Zhiyuan Liu, Hao Peng, Lei Bai, Wanli Ouyang, Yu Cheng, Bowen Zhou, and Ning Ding. The entropy mechanism of reinforcement learning for reasoning language models. arXiv preprint arXiv:2505.22617, 2025. URL <https://arxiv.org/abs/2505.22617>.
- [7] Xingyu Dang, Christina Baek, J. Zico Kolter, and Aditi Raghunathan. Assessing diversity collapse in reasoning. In *ICLR Workshop on Scaling Self-Improving Foundation Models without Human Supervision*, 2025. URL <https://openreview.net/forum?id=AMiKsHLjQh>.
- [8] Hanze Dong, Wei Xiong, Deepanshu Goyal, Yihan Zhang, Winnie Chow, Rui Pan, Shizhe Diao, Jipeng Zhang, Kashun Shum, and Tong Zhang. RAFT: Reward rAnked FineTuning for generative foundation model alignment. *Transactions on Machine Learning Research*, 2023. URL <https://arxiv.org/abs/2304.06767>.
- [9] Anil R. Doshi and Oliver P. Hauser. Generative AI enhances individual creativity but reduces the collective diversity of novel content. *Science Advances*, 10(28):eadn5290, 2024. doi: 10.1126/sciadv.adn5290.
- [10] Sebastian Farquhar, Jannik Kossen, Lorenz Kuhn, and Yarin Gal. Detecting hallucinations in large language models using semantic entropy. *Nature*, 630(8017):625–630, 2024. doi: 10.1038/s41586-024-07421-0.
- [11] Saibo Geng, Martin Josifoski, Maxime Peyrard, and Robert West. Grammar-constrained decoding for structured NLP tasks without finetuning. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pp. 10932–10952, Singapore, 2023. Association for Computational Linguistics. doi: 10.18653/v1/2023.emnlp-main.674. URL <https://aclanthology.org/2023.emnlp-main.674/>.
- [12] Daya Guo, Dejian Yang, Haowei Zhang, Junxiao Song, Peiyi Wang, Qihao Zhu, Runxin Xu, Ruoyu Zhang, Shirong Ma, Xiao Bi, et al. DeepSeek-R1 incentivizes reasoning in LLMs through reinforcement learning. *Nature*, 645(8081):633–638, 2025. doi: 10.1038/s41586-025-09422-z.
- [13] Tuomas Haarnoja, Aurick Zhou, Pieter Abbeel, and Sergey Levine. Soft actor-critic: Off-policy maximum entropy deep reinforcement learning with a stochastic actor. In Jennifer Dy and Andreas Krause (eds.), *Proceedings of the 35th International Conference on Machine Learning (ICML)*, volume 80 of *Proceedings of Machine Learning Research*, pp. 1861–1870. PMLR, 2018. URL <https://proceedings.mlr.press/v80/haarnoja18b.html>.

-
- [14] Tuomas Haarnoja, Aurick Zhou, Kristian Hartikainen, George Tucker, Sehoon Ha, Jie Tan, Vikash Kumar, Henry Zhu, Abhishek Gupta, Pieter Abbeel, and Sergey Levine. Soft actor-critic algorithms and applications. *arXiv preprint arXiv:1812.05905*, 2018. URL <https://arxiv.org/abs/1812.05905>.
- [15] Edward J. Hu, Moksh Jain, Eric Elmoznino, Younesse Kaddar, Guillaume Lajoie, Yoshua Bengio, and Nikolay Malkin. Amortizing intractable inference in large language models. In *International Conference on Learning Representations*, 2024. URL <https://arxiv.org/abs/2310.04363>.
- [16] Robert Kirk, Ishita Mediratta, Christoforos Nalmpantis, Jelena Luketina, Eric Hambro, Edward Grefenstette, and Roberta Raileanu. Understanding the effects of RLHF on LLM generalisation and diversity. In *International Conference on Learning Representations*, 2024. URL <https://openreview.net/forum?id=PXD3FAVHJT>.
- [17] Joel Lehman and Kenneth O. Stanley. Abandoning objectives: Evolution through the search for novelty alone. *Evolutionary Computation*, 19(2):189–223, 2011.
- [18] Sergey Levine. Reinforcement learning and control as probabilistic inference: Tutorial and review. *arXiv preprint arXiv:1805.00909*, 2018. URL <https://arxiv.org/abs/1805.00909>.
- [19] Chenyi Li, Yuan Zhang, Bo Wang, Guoqing Ma, Wei Tang, Haoyang Huang, and Nan Duan. SetPO: Set-level policy optimization for diversity-preserving LLM reasoning, 2026. URL <https://arxiv.org/abs/2602.01062>.
- [20] Long Li, Zhijian Zhou, Jiaran Hao, Jason Klein Liu, Yanting Miao, Wei Pang, Xiaoyu Tan, Wei Chu, Zhe Wang, Shirui Pan, Chao Qu, and Yuan Qi. The choice of divergence: A neglected key to mitigating diversity collapse in reinforcement learning with verifiable reward, 2025. URL <https://arxiv.org/abs/2509.07430>.
- [21] Xiaozhe Li, Yang Li, Xinyu Fang, Shengyuan Ding, Peiji Li, Yongkang Chen, Yichuan Ma, Tianyi Lyu, Linyang Li, Dahua Lin, Qipeng Guo, Qingwen Liu, and Kai Chen. Beyond mode collapse: Distribution matching for diverse reasoning, 2026. URL <https://arxiv.org/abs/2605.19461>.
- [22] Hunter Lightman, Vineet Kosaraju, Yuri Burda, Harrison Edwards, Bowen Baker, Teddy Lee, Jan Leike, John Schulman, Ilya Sutskever, and Karl Cobbe. Let’s verify step by step. In *The Twelfth International Conference on Learning Representations (ICLR)*, 2024.
- [23] Zichen Liu, Changyu Chen, Wenjun Li, Penghui Qi, Tianyu Pang, Chao Du, Wee Sun Lee, and Min Lin. Understanding R1-Zero-like training: A critical perspective. In *Conference on Language Modeling (COLM)*, 2025. URL <https://arxiv.org/abs/2503.20783>.
- [24] Jean-Baptiste Mouret and Jeff Clune. Illuminating search spaces by mapping elites, 2015. URL <https://arxiv.org/abs/1504.04909>.
- [25] Peter S. Park, Philipp Schoenegger, and Chongyang Zhu. Diminished diversity-of-thought in a standard large language model. *Behavior Research Methods*, 56(6):5754–5770, 2024. doi: 10.3758/s13428-023-02307-x.
- [26] David Rolnick, Arun Ahuja, Jonathan Schwarz, Timothy P. Lillicrap, and Greg Wayne. Experience replay for continual learning. In *Advances in Neural Information Processing Systems*, volume 32, 2019. URL <https://arxiv.org/abs/1811.11682>.
- [27] Zhihong Shao, Peiyi Wang, Qihao Zhu, Runxin Xu, Junxiao Song, Xiao Bi, Haowei Zhang, Mingchuan Zhang, Y. K. Li, Y. Wu, and Daya Guo. DeepSeekMath: Pushing the limits of mathematical reasoning in open language models. *arXiv preprint arXiv:2402.03300*, 2024.
- [28] Abhijeet Sinha, Sundari Elango, and Dianbo Liu. Expected return causes outcome-level mode collapse in reinforcement learning and how to fix it with inverse probability scaling, 2026. URL <https://arxiv.org/abs/2601.21669>.

-
- [29] Charlie Snell, Jaehoon Lee, Kelvin Xu, and Aviral Kumar. Scaling LLM test-time compute optimally can be more effective than scaling model parameters, 2024. URL <https://arxiv.org/abs/2408.03314>.
- [30] Yuda Song, Julia Kempe, and Remi Munos. Outcome-based exploration for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.06941>.
- [31] Taylor Sorensen, Jared Moore, Jillian Fisher, Mitchell L. Gordon, Niloofar Miresghallah, Christopher Michael Rytting, Andre Ye, Liwei Jiang, Ximing Lu, Nouha Dziri, Tim Althoff, and Yejin Choi. Position: A roadmap to pluralistic alignment. In *Proceedings of the 41st International Conference on Machine Learning*, volume 235 of *Proceedings of Machine Learning Research*, pp. 46280–46302. PMLR, 2024. URL <https://proceedings.mlr.press/v235/sorensen24a.html>.
- [32] Angelina Wang, Jamie Morgenstern, and John P. Dickerson. Large language models that replace human participants can harmfully misportray and flatten identity groups. *Nature Machine Intelligence*, 7:400–411, 2025. doi: 10.1038/s42256-025-00986-z.
- [33] Xuezhi Wang, Jason Wei, Dale Schuurmans, Quoc V. Le, Ed H. Chi, Sharan Narang, Aakanksha Chowdhery, and Denny Zhou. Self-consistency improves chain of thought reasoning in language models. In *The Eleventh International Conference on Learning Representations (ICLR)*, 2023. URL <https://openreview.net/forum?id=1PL1NIMMrw>.
- [34] Jian Yao, Ran Cheng, Xingyu Wu, Jibin Wu, and Kay Chen Tan. Diversity-aware policy optimization for large language model reasoning, 2025. URL <https://arxiv.org/abs/2505.23433>.
- [35] Qiyang Yu, Zheng Zhang, Ruofei Zhu, Yufeng Yuan, Xiaochen Zuo, Yu Yue, Tiantian Fan, Gaohong Liu, Lingjun Liu, Xin Liu, Haibin Lin, Zhiqi Lin, Bole Ma, Guangming Sheng, Yuxuan Tong, Chi Zhang, Mofan Zhang, Wang Zhang, Hang Zhu, Jinhua Zhu, Jiaze Chen, Jiangjie Chen, Chengyi Wang, Hongli Yu, Weinan Dai, Yuxuan Song, Xiangpeng Wei, Hao Zhou, Jingjing Liu, Wei-Ying Ma, Ya-Qin Zhang, Lin Yan, Mu Qiao, Yonghui Wu, and Mingxuan Wang. DAPO: An open-source LLM reinforcement learning system at scale, 2025. URL <https://arxiv.org/abs/2503.14476>.
- [36] Yang Yue, Zhiqi Chen, Rui Lu, Andrew Zhao, Zhaokai Wang, Yang Yue, Shiji Song, and Gao Huang. Does reinforcement learning really incentivize reasoning capacity in LLMs beyond the base model?, 2025. URL <https://arxiv.org/abs/2504.13837>.
- [37] Xuekai Zhu, Daixuan Cheng, Dinghuai Zhang, Hengli Li, Kaiyan Zhang, Che Zhou, Youbang Qi, Ermo Ding, Yuxin Cheng, and Bowen Zhou. FlowRL: Matching reward distributions for LLM reasoning, 2025. URL <https://arxiv.org/abs/2509.15207>.
- [38] Brian D. Ziebart, Andrew L. Maas, J. Andrew Bagnell, and Anind K. Dey. Maximum entropy inverse reinforcement learning. In *Proceedings of the Twenty-Third AAAI Conference on Artificial Intelligence (AAAI)*, pp. 1433–1438. AAAI Press, 2008. URL <https://cdn.aaai.org/AAAI/2008/AAAI08-227.pdf>.

A WHY GRPO COLLAPSES AND xGRPO DOES NOT AFTER COVERAGE

Main-body link. This appendix supplies the formal claims invoked in Sections 4 and 7.

This appendix separates two questions that are easy to conflate: whether a binary-reward group update preserves ratios among already correct modes, and whether xGRPO preserves every correct mode that exists. The first admits an unconditional answer in a finite categorical model. The second necessarily requires a coverage assumption: no method whose bank contains only executed, validator-positive objects can protect a mode that is never sampled or constructed. We therefore prove support preservation first and state full no-collapse as a coverage corollary.

A.1 CATEGORICAL REDUCTION AND THE GRPO MEAN UPDATE

Fix one prompt. Let the finite response categories be $\mathcal{A} = \mathcal{C} \dot{\cup} \mathcal{N}$, where the $m \geq 2$ categories in $\mathcal{C}$ are correct execution modes and the $n \geq 1$ categories in $\mathcal{N}$ are incorrect. Let $p_a = \text{softmax}(z)_a$, $P = \sum_{c \in \mathcal{C}} p_c$, and $q_c = p_c/P$. Thus q is exactly the conditional correct-mode distribution $p_\theta^+(\cdot | x)$ of Section 2, and P is the correct mass $P_\theta^+(x)$; it is not the detached count predictor $\hat{q}_x$ of Section 4, which the policy never differentiates through.

Draw a group $A_1, \dots, A_G \stackrel{\text{iid}}{\sim} p$ with $G \geq 2$, write $R_i = \mathbf{1}[A_i \in \mathcal{C}]$, and let $M = \sum_i R_i$. Both GRPO variants can be written, at the on-policy point where the PPO ratio equals one, as

$$\hat{g}_G = \frac{1}{G} \sum_{i=1}^G w(M) \left(R_i - \frac{M}{G} \right) \nabla_z \log p_{A_i}. \quad (1)$$

For Dr.GRPO, $w(M) = 1$ after absorbing the shared positive length scale. For binary-reward GRPO, $w(M)$ is the reciprocal group reward standard deviation on mixed groups. In either case $w(\ell) > 0$ for $\ell \in \{1, \dots, G-1\}$; its value on all-equal groups is irrelevant because the centered advantage is zero. We take explicit reference KL to be zero, as in the experiments.

Lemma A.1 (A group baseline does not remove frequency amplification). *For $0 < P < 1$, the expected group update is*

$$\mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P, \quad c_G(P) = \frac{\mathbb{E}[w(M)M(G-M)]}{G^2 P(1-P)} > 0. \quad (2)$$

For Dr.GRPO, $c_G(P) = (G-1)/G$. Hence reward centering and binary reward standardization change the speed, but not the expected direction, of the binary-reward policy gradient.

Proof. Condition on the complete binary reward vector, whose sum is M . The score identities give

$$\mathbb{E}[\nabla_z \log p_{A_i} | R_i = 1] = \nabla_z \log P, \quad \mathbb{E}[\nabla_z \log p_{A_i} | R_i = 0] = \nabla_z \log(1-P).$$

There are M correct rows with centered reward $(G-M)/G$ and $G-M$ incorrect rows with centered reward $-M/G$. Therefore the conditional expectation of the unaveraged sum in (1) is

$$w(M) \frac{M(G-M)}{G} (\nabla_z \log P - \nabla_z \log(1-P)) = w(M) \frac{M(G-M)}{GP(1-P)} \nabla_z P.$$

The outer factor $1/G$ and expectation over $M \sim \text{Binomial}(G, P)$ yield (2). For $w \equiv 1$, $\mathbb{E}[M(G-M)] = G(G-1)P(1-P)$, proving the Dr.GRPO value. $\square$

A.2 WINNER-TAKE-ALL COLLAPSE UNDER GRPO AND DR.GRPO

Consider the infinitesimal expected-update flow $\dot{z} = \mathbb{E}[\hat{g}_G] = c_G(P) \nabla_z P$. This is the clean mean-field limit of fresh on-policy groups and the first-order PPO update. It deliberately removes sampling noise; the result below therefore identifies a structural instability rather than blaming finite batches.

Theorem A.2 (Generic binary-reward GRPO mode collapse). *Start from finite logits with $0 < P(0) < 1$. If the conditional correct-mode distribution $q(0)$ has a unique largest coordinate c^* , then*

$$P(t) \longrightarrow 1, \quad q_{c^*}(t) \longrightarrow 1, \quad q_c(t) \longrightarrow 0 \quad (c \neq c^*).$$

Thus both GRPO and Dr.GRPO converge to a single correct execution mode for generic initial conditions, even though every mode in $\mathcal{C}$ has exactly the same reward.

Proof. For a correct mode c and incorrect category a ,

$$\dot{z}_c = c_G(P)p_c(1 - P), \quad \dot{z}_a = -c_G(P)p_aP.$$

Also $\dot{P} = c_G(P)\|\nabla_z P\|_2^2 > 0$. If P converged to an interior value, continuity and positivity of c_G , together with

$$\|\nabla_z P\|_2^2 \geq P^2(1 - P)^2 \left(\frac{1}{m} + \frac{1}{n} \right),$$

would keep $\dot{P}$ bounded away from zero, a contradiction. Hence $P \rightarrow 1$.

Define the effective optimization time

$$\tau(t) = \int_0^t c_G(P(s))P(s)(1 - P(s)) ds.$$

If $\tau(\infty) < \infty$, every correct and incorrect logit would have finite total variation, because the absolute velocity of each is at most $\dot{\tau}$. All logits would then converge to finite values, contradicting $P \rightarrow 1$. Therefore $\tau(t) \rightarrow \infty$.

On the correct simplex, changing from t to τ gives the replicator flow

$$\frac{dq_c}{d\tau} = q_c \left(q_c - \sum_{d \in \mathcal{C}} q_d^2 \right), \quad \frac{d}{d\tau} \log \frac{q_c}{q_d} = q_c - q_d. \quad (3)$$

The initially unique maximum remains the unique maximum, and every one of its log-ratios increases. The Lyapunov quantity $\sum_c q_c^2$ is nondecreasing, since its derivative is twice the variance of the coordinate values under q . Every limit point of (3) is therefore uniform on its positive support. Because $q_{c^*} \geq 1/m$ and its ratio to every other coordinate stays strictly greater than one, no multi-coordinate uniform limit is possible. The only limit point is the vertex $q_{c^*} = 1$. $\square$

Corollary A.3 (Ties are nongeneric). *If exactly $k < m$ correct modes tie for the largest initial probability, symmetry preserves the tie and the flow converges to the uniform distribution on those k modes while eliminating the rest. Only the exactly uniform initialization over all m correct modes avoids collapse. Any arbitrarily small perturbation that creates a unique maximum reduces to Theorem A.2.*

Finite groups make this instability operationally harsher: a mode absent from a group has no score carrier, all-correct groups have zero task advantage, and sampling noise almost surely breaks exact ties. These facts are not needed for the theorem.

A.3 WHY THE XGRPO TERMS REVERSE THE INSTABILITY

The open-set actor signal has a direct maximum-entropy interpretation before clipping and count approximation.

Lemma A.4 (Centered surprisal is the execution-mode entropy gradient). *Let $C \sim q$ on a known correct support and $\mathcal{H}(q) = -\sum_c q_c \log q_c$. Then*

$$\mathbb{E}_{C \sim q} [(-\log q_C - \mathcal{H}(q)) \nabla_z \log q_C] = \nabla_z \mathcal{H}(q).$$

Thus, when the detached predictor equals the current correct-mode distribution, the centered rare-mode advantage is an unbiased entropy-gradient estimator.

Proof. The baseline vanishes by the score identity. For the remaining term,

$$\sum_c q_c (-\log q_c) \nabla_z \log q_c = -\sum_c (\log q_c) \nabla_z q_c = \nabla_z \mathcal{H}(q),$$

because $\sum_c \nabla_z q_c = 0$ supplies the missing constant in the derivative of $-q_c \log q_c$. $\square$

The implemented pseudocount, unseen bucket, and surprisal clip make this a robust surrogate rather than an exact identity. The one-time new-mode term adds total credit ν independent of the number of

duplicates once a new key appears. Neither signal can assign gradient to a never-observed zero-carrier mode; that is why verified replay and the coverage condition below matter. The next result needs only the balance term; it does not count any additional restoring pressure from rare-mode or verified-mass replay. We index time by mean updates acting on this prompt, absorbing fixed round-robin frequency and replay scaling into the effective coefficient $a(t)$; the categorical abstraction excludes cross-prompt parameter interference.

Let a verified bank contain $m \geq 2$ modes, let $u_c = 1/m$, and let q be the model distribution normalized over those bank modes. In the categorical projection, descending the implemented balance loss $L(q) = \text{KL}(u||q)$ contributes $a(t)(u - q)$ to the mode logits. Binary GRPO contributes the amplifying term $b(t)q$, where $b(t) = c_G(P)P(1 - P)$ in the mean flow. Terms shared by every correct-mode logit are denoted $d(t)\mathbf{1}$ and do not change q .

Theorem A.5 (Verified balance makes the uniform bank the unique attractor). *Suppose that after some time T the bank is fixed and scheduled infinitely often, and its mean conditional logit dynamics are*

$$\dot{z} = b(t)q + a(t)(u - q) + d(t)\mathbf{1}. \quad (4)$$

If $a(t) > b(t)$ eventually and $\int_T^\infty (a(t) - b(t)) dt = \infty$, then

$$\text{KL}(u||q(t)) \longrightarrow 0, \quad q(t) \longrightarrow u.$$

Consequently every bank mode retains asymptotic probability $1/m$ and conditional mode collapse is impossible on that bank.

Proof. Let $\lambda(t) = a(t) - b(t)$. Since $u = (1/m)\mathbf{1}$, Equation (4) is, up to a common-logit shift,

$$\dot{z} = -\lambda(t)(q - u) + \tilde{d}(t)\mathbf{1}.$$

For $L(z) = \text{KL}(u||\text{softmax}(z))$, $\nabla_z L = q - u$ and $(q - u)^\top \mathbf{1} = 0$. Hence

$$\dot{L} = -\lambda(t)\|q - u\|_2^2 \leq 0. \quad (5)$$

Use replay time $s = \int_T^t \lambda(r) dr$, which diverges by assumption. Equation (5) is ordinary gradient flow for L modulo the irrelevant common-logit direction. Its sublevel sets are compact in the open simplex because $\text{KL}(u||q) \rightarrow \infty$ at the boundary, and its only stationary distribution is $q = u$. Therefore $q(t) \rightarrow u$ and $L(t) \rightarrow 0$. $\square$

Corollary A.6 (xGRPO no-collapse guarantee after verified coverage). *Assume the categorical mean-flow model above and:*

1. *every correct execution mode enters and remains in the verified bank by a finite time—so the protected support fits within replay capacity;*
2. *deterministic global replay visits that bank infinitely often;*
3. *the balance controller is active with reference entropy $h_{\text{ref}} > 0$ and tracks bank entropy through $a(t) = a_0 h_{\text{ref}} / h_{\text{ema}}(t)$; and*
4. *correct mass approaches one, so the GRPO amplification $b(t) = c_G(P)P(1 - P)$ approaches zero.*

Then $q(t) \rightarrow u$ on the full correct support. In particular,

$$\min_{c \in \mathcal{C}} q_c(t) \longrightarrow \frac{1}{m}, \quad \mathcal{H}(q(t)) \longrightarrow \log m,$$

so mode collapse cannot emerge asymptotically.

Proof. Because $0 < h_{\text{ema}}(t) \leq \log m$ on eligible replay observations, $a(t) \geq a_0 h_{\text{ref}} / \log m =: a_{\min} > 0$. Since $b(t) \rightarrow 0$, eventually $a(t) - b(t) \geq a_{\min}/2$, whose integral over infinitely many replay visits diverges. Theorem A.5 applies. Moreover, if entropy attempted to approach zero before that regime, $h_{\text{ema}} \rightarrow 0$ would make $a(t) \rightarrow \infty$ while $b(t)$ remains bounded, so the boundary cannot be an attracting limit. $\square$

Remark A.7 (Exact scope of the guarantee). If the verified bank covers only $\mathcal{B} \subsetneq \mathcal{C}$ —including when the true support exceeds the implementation’s capacity 16—the theorem guarantees non-collapse and asymptotic balance on $\mathcal{B}$, not discovery of $\mathcal{C} \setminus \mathcal{B}$. The open-set unseen bucket, first-discovery credit, and separated-support actuator are mechanisms for satisfying coverage; they are not a catalogue oracle. Likewise, the proof concerns realizable categorical mean dynamics with recurrent replay. It does not claim global convergence for an arbitrary nonconvex language model. This distinction is what makes the guarantee falsifiable rather than rhetorical.

B EXTERNAL PYTHON VERIFICATION CONTRACT

Main-body link. This appendix expands the Python Factors contract in Section 3 and its limitation in Section 9.

A Python response is parsed in expression mode and must be a lambda with one argument named n . Accepted nodes are integer constants, n , bounded arithmetic, comparisons, Boolean operators, unary operators, and conditional expressions. Calls, imports, attributes, comprehensions, containers, subscripts, assignment, loops, and every other identifier are rejected before execution. Candidate length and AST size are bounded.

The parent sends restricted source and trusted JSON task data to a persistent external interpreter over JSON lines. The worker reparses, removes builtins, evaluates the lambda, and invokes it on all frozen inputs under a per-request timer. It returns only validity, the integer vector, and its key. Timeout, broken pipe, malformed output, non-integer values, or divisor failure returns no key. The parent can restart a failed worker.

For inputs $[6, 10, 15]$,

```
lambda n: 2 if n % 2 == 0 else 3
```

returns $(2, 2, 3)$, while

```
lambda n: 3 if n % 3 == 0 else 5
```

returns $(3, 5, 3)$. Both verify and receive different behavior keys.

C DOMAIN PROMPTS

Main-body link. This appendix reproduces the exact prompt the policy receives in each of the five MODEBENCH domains of Section 3 and Table 1.

Every domain is single-turn: one system message, one user message, and no few-shot examples. The user message is the dataset problem field; the wrapper is the chat template the launcher exported through `OAT_ZERO_PROMPT_TEMPLATE`, defined in `src/oat_drgrepo/templates.py`. Both arms of every domain use the identical string, and the same template is applied at training and at evaluation, so the prompt is never a difference between arms. Each block below is the first row of that cohort’s frozen evaluation split, rendered by that template; `ops/check_paper_domain_prompts.py` regenerates these blocks from the datasets and fails closed if the typeset text drifts from them. Lines longer than the column are wrapped here with a two-space continuation indent; joining each indented continuation to its predecessor with one space recovers the string byte for byte, and the generator asserts that round trip. The only real newlines are the ones inside the `PantryPlan` pantry listing.

C.1 GRAPH COLORING

Template `qwen_boxed`; 384 training and 128 evaluation rows in `var/data/graph_coloring_modebench_v2`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Color this graph with vertices 1 through 6. The undirected edges are: (1,2), (1,3), (3,5),
(3,6), (4,6), (5,6). Assign each vertex one color from {1, 2, 3} so that no edge has the
same color on both endpoints. A partial coloring is 2??1?2, where ? means uncolored.
Keep every shown digit fixed. There are exactly 3 question marks. Return exactly one
final answer and no explanation: exactly 3 digits, each one 1, 2, or 3, for the missing
positions from left to right, inside \boxed{ }.<|im_end|>
<|im_start|>assistant
```

C.2 COUNTDOWN

Template `qwen_boxed`; 384 training and 128 evaluation rows in `var/data/exact_countdown_easy3_probe`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Using the numbers [3, 6, 9], create an arithmetic expression that equals 18. Use each
given number exactly once. You may use +, -, *, /, and parentheses. Return exactly one
final answer and no explanation: the expression inside \boxed{ }.<|im_end|>
<|im_start|>assistant
```

C.3 PYTHON FACTORS

Template `qwen_boxed`; 384 training and 128 evaluation rows in `var/data/python_factor_modebench_v1`.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
Write one pure Python function with the exact form lambda n: EXPR. An external Python tool
will call it once for each n in [18, 82, 91, 93]. For every call, return an integer d
satisfying 1 < d < n and n % d == 0. Different valid return vectors are different
solution modes. You may use integer literals, n, +, -, *, /, //, %, comparisons, Boolean
operators, and conditional expressions; calls, imports, attributes, containers, and
other names are forbidden. Return exactly the one-line lambda inside \boxed{ } and no
explanation.<|im_end|>
<|im_start|>assistant
```

C.4 MATHIR ACTION MENU

Template qwen_boxed; 384 training and 128 evaluation rows in var/data/mathir_action_menu_v1.

```
<|im_start|>system
Return only the final answer inside \boxed{}. Do not explain.<|im_end|>
<|im_start|>user
MathIR linear-menu-v1. Solve for x by choosing an executable action sequence. Every
selected action is applied exactly to BOTH sides of the current equation, followed by
exact simplification. Bindings: a=2, b=-9, c=-1. Initial equation: x/a + b = c. Action
menu: A means div(a); B means add(b); C means sub(b); D means sub(c); E means
sub(mul(a,b)); F means mul(a). Choose 1-4 action IDs. Illegal operations, repeated
states, and paths that do not finish with x isolated are rejected. Return only the IDs
separated by semicolons inside the answer box; for example formatting only: \boxed{A;C}.
Do not return x's numeric value.<|im_end|>
<|im_start|>assistant
```

C.5 PANTRYPLAN

Template qwen_pantry_support_mask; 384 training and 128 evaluation rows in var/data/pantry_plan_modebench_v2.

```
<|im_start|>system
Return only six binary digits and no other text. Each digit maps to the corresponding
Pantry row in printed order. A trusted environment will choose exact quantities on
precisely the selected rows.<|im_end|>
<|im_start|>user
Create one feasible high fiber snack from this frozen pantry.
Nutrition values are per 100 g. Quantities are exact integer grams.

Pantry:
- almonds: available=100g, step=25g, minimum-if-used=50g; energy=584kcal, protein=21.5g,
  fiber=10.8g, sodium=0.0mg; tags=tree_nut
- banana: available=125g, step=25g, minimum-if-used=50g; energy=97.0kcal, protein=0.74g,
  fiber=4.62g, sodium=0.0mg; tags=none
- carrots: available=150g, step=25g, minimum-if-used=50g; energy=45.0kcal, protein=0.941g,
  fiber=3.1g, sodium=86.6mg; tags=none
- grape_tomatoes: available=100g, step=25g, minimum-if-used=50g; energy=27.0kcal,
  protein=0.83g, fiber=2.1g, sodium=6.0mg; tags=none
- navel_orange: available=150g, step=25g, minimum-if-used=50g; energy=47.0kcal,
  protein=0.91g, fiber=2.0g, sodium=9.0mg; tags=none
- sunflower_seeds: available=125g, step=25g, minimum-if-used=50g; energy=571kcal,
  protein=18.9g, fiber=7.22g, sodium=0.0mg; tags=none

Use 2 to 4 ingredients.
Forbidden tags: none
Targets:
- mass_g: minimum 125, maximum 200
- energy_kcal: minimum 239.4, maximum 432.25
- protein_g: minimum 6.423
- fiber_g: minimum 3.478
- sodium_mg: maximum 37.6

Choose the ingredient support as a six-bit mask in the exact Pantry row order. Bit 1
includes that row and bit 0 excludes it.<|im_end|>
<|im_start|>assistant
```

D DATASET AND IDENTITY AUDITS

Main-body link. This appendix audits the materialization and identity claims in Section 3 and Table 1.

1026
1027
1028
1029
1030
1031
1032
1033
1034
1035
1036
1037
1038
1039
1040
1041
1042
1043
1044
1045
1046
1047
1048
1049
1050
1051
1052
1053
1054
1055
1056
1057
1058
1059
1060
1061
1062
1063
1064
1065
1066
1067
1068
1069
1070
1071
1072
1073
1074
1075
1076
1077
1078
1079

	Python factors	MathIR action menu
Training rows	384	384
Evaluation rows	128	128
Exact modes per prompt	16–3,600	5
Training-row SHA-256	bcfa9acc...37cf7	bfc98362...5d3835
Evaluation-row SHA-256	be0f621c...94d183	df5783dd...328b96
Gold catalogue supplied to policy	no	no
Training-bank initialization	empty	empty

Table 3: Deterministic materialization identities. For every Python row, two independently executed functions must verify with different keys. Every MathIR certified route is executed by the same interpreter used for model outputs.

PantryPlan audit. The deterministic v2 release contains 384 train, 64 development, and 128 evaluation prompts across four formulation families; exact exhaustive support counts range from 8 to 45. The reported comparison uses the full 384-prompt training split and the fixed 128-row evaluation split, whose 2,328 feasible supports average 18.1875 per prompt. The train/evaluation row hashes are a4883c5e...a757f and 5d81ba8d...5923. The terminal audit status is `pass` with decision `pantry_terminal_eligible`. Correctness checks exact inventory, serving steps, mass, nutrient bounds, and dietary tags; identity is ingredient support. The benchmark does not claim that a verified allocation is a recipe, palatable, cookable, or safe.

E ALGORITHMIC DETAILS

Main-body link. This appendix formalizes the state transition summarized by Figure 3 and Section 4. For each prompt x , let $\mathcal{T}_x$ be the historical on-policy key *tally* used by the semantic actor (distinct from $\mathcal{C}_x$, the set of correct modes that exist), $\mathcal{R}_x$ the capacity- M replay-exemplar state, τ the global replay cursor, and Γ the three controller states. The separation $\mathcal{T}_x \neq \mathcal{R}_x$ matters only when the optional support proposal is enabled.

Algorithm 1 One xGRPO optimizer update

Require: Policy π_θ ; prompt x and verifier specification s_x ; group size G ; states $(\mathcal{T}, \mathcal{R}, \tau, \Gamma)$
Ensure: Atomically updated $(\theta', \mathcal{T}', \mathcal{R}', \tau', \Gamma')$

- 1: Freeze $\mathcal{T}_x^- \leftarrow \mathcal{T}_x$ and $\mathcal{R}_x^- \leftarrow \mathcal{R}_x$; read coefficients $(\beta_t, \mu_t, \alpha_t)$ from Γ
- 2: Draw the on-policy group $\mathcal{G} = \{y_i \sim \pi_\theta(\cdot | x)\}_{i=1}^G$
- 3: **for** $i = 1, \dots, G$ **do**
- 4: Execute $\tilde{c}_i \leftarrow V(y_i, s_x)$ and observe task reward r_i
- 5: Set $c_i \leftarrow \tilde{c}_i$ iff $r_i > 0 \wedge \tilde{c}_i \neq \perp$; otherwise set $c_i \leftarrow \perp$
- 6: **end for**
- 7: Compute centered Dr.GRPO task advantages A_i^{task} from $\{r_j\}_{j=1}^G$
- 8: **for** $i = 1, \dots, G$ with $c_i \neq \perp$ **do**
- 9: Form the detached leave-one-out open-set predictor $\hat{q}_{x,-i}$ from $\mathcal{T}_x^-$ and $\{c_j : j \neq i, c_j \neq \perp\}$
- 10: Compute predictive-centered rarity A_i^{rare} and split one first-seen credit A_i^{new} across equal new keys
- 11: **end for**
- 12: Set $A_i \leftarrow A_i^{\text{task}} + \mathbf{1}[c_i \neq \perp](A_i^{\text{rare}} + A_i^{\text{new}})$
- 13: **if** the registered singleton gate is eligible **then**
- 14: Construct at most one domain transformation $\tilde{y}$; re-execute it with V
- 15: **if** $V(\tilde{y}, s_x)$ is a new verified key **then**
- 16: Add its exemplar only to $\mathcal{R}_x$; never add it to $\mathcal{G}$, PPO, or $\mathcal{T}_x$
- 17: **end if**
- 18: **end if**
- 19: After all group scores are fixed, commit each neutral verified c_i to $\mathcal{T}_x$ and its first exemplar to $\mathcal{R}_x$
- 20: $(z, \tau') \leftarrow \text{NEXTBANK}(\{\mathcal{R}_u\}_u, \tau)$ by deterministic global round robin
- 21: **if** a replay bank $\mathcal{R}_z$ exists **then**
- 22: For each retained exemplar e_c , compute the length-normalized score $s_c = |e_c|^{-1} \log \pi_\theta(e_c | z)$
- 23: $L_{\text{mass}} \leftarrow -|\mathcal{R}_z|^{-1} \sum_{c \in \mathcal{R}_z} s_c$
- 24: $L_{\text{balance}} \leftarrow \mathbf{1}[|\mathcal{R}_z| > 1] \text{KL}(U(\mathcal{R}_z) \| \text{softmax}_c s_c)$
- 25: **else**
- 26: $L_{\text{mass}}, L_{\text{balance}} \leftarrow 0$
- 27: **end if**
- 28: $w_{\text{rep}} \leftarrow (G - 1)/G^2$ for one replay pseudo-group per G on-policy rows
- 29: $\theta' \leftarrow \text{UPDATE}(\theta, L_{\text{PPO}}(A) + w_{\text{rep}}[\mu_t L_{\text{mass}} + \alpha_t L_{\text{balance}}])$
- 30: Update only eligible controller EMAs/references to obtain Γ' for the next update
- 31: Checkpoint $(\theta', \mathcal{T}', \mathcal{R}', \tau', \Gamma')$ as one atomic state

The pre-update controller values act throughout the current transition; their updated values first act on the next one. The base objective omits the optional singleton block; the actuator variant enables it with separated support. Neither version uses a gold mode catalogue, direct token entropy, or the historical prompt-local bank-entropy actor channel (`bank_alpha=0`). Fixed doses, warmups, and capacities are experimental constants in Section 6, not steps of the algorithm.

F PER-SEED PASS-12 RESULTS

Main-body link. This appendix expands the terminal five-seed means in Section 7 and Table 2.

Domain (pass)	Method	seed	pass@8	mean@8	distinct@8	pass@1
Graph (12)	Dr.GRPO	43	.312	.312	.312	.312
		44	.320	.316	.320	.320
		45	.320	.317	.320	.320
		46	.320	.315	.320	.312
		47	.352	.352	.352	.352
	xGRPO	43	.971	.433	2.406	.484
		44	.955	.394	2.389	.477
		45	.971	.402	2.461	.414
		46	.949	.415	2.439	.422
		47	.957	.391	2.334	.516
Countdown (12)	Dr.GRPO	43	.438	.437	.445	.438
		44	.602	.584	.725	.586
		45	.648	.635	.648	.633
		46	.656	.656	.691	.656
		47	.625	.622	.625	.625
	xGRPO	43	.705	.581	1.803	.586
		44	.701	.618	2.033	.625
		45	.715	.629	1.484	.641
		46	.672	.614	2.131	.641
		47	.670	.610	2.014	.609
Python (12)	Dr.GRPO	43	.172	.172	.172	.172
		44	.172	.172	.172	.172
		45	.172	.172	.172	.172
		46	.172	.172	.172	.172
		47	.172	.172	.172	.172
	xGRPO	43	1.000	.993	2.619	.992
		44	1.000	.980	2.199	1.000
		45	.211	.201	.211	.203
		46	1.000	.973	2.713	1.000
		47	.227	.222	.227	.219
MathIR (12)	Dr.GRPO	43	.680	.625	.680	.625
		44	.602	.572	.602	.578
		45	.738	.720	.738	.719
		46	.781	.781	.781	.781
		47	.531	.520	.531	.523
	xGRPO	43	.826	.733	.850	.789
		44	.879	.696	.969	.727
		45	.914	.784	.988	.812
		46	.807	.675	.846	.688
		47	.908	.789	.977	.812

Table 4: Seed-level values underlying Table 2. Each block is at the terminal pass 12, reached by all five seeds of both arms. Values are rounded to three decimals.

Domain (pass)	Method	seed	pass@8	mean@8	distinct@8	pass@1
PantryPlan (12)	Dr.GRPO	43	.703	.543	1.090	.547
		44	.531	.531	.531	.531
		45	.531	.531	.531	.531
		46	.484	.484	.484	.484
		47	.531	.531	.531	.531
	xGRPO	43	.863	.584	2.211	.641
		44	.814	.581	1.885	.633
		45	.857	.572	1.971	.617
		46	.893	.552	2.504	.570
		47	.918	.544	2.359	.562

Table 5: Completed PantryPlan terminal rows. All five paired seeds end at pass 12; values are rounded to three decimals. This domain shares the seed set and reporting rule of the other four and, like them, is reported on its own rather than pooled into a single effect.

G TERMINAL MATHIR CAUSAL ROWS

Main-body link. This appendix gives the six seed rows behind the causal result in Section 7 and Table 6.

arm	seed	greedy	mean@8	pass@8	distinct@8	bank support
actuator off	43	.7500	.7207	.8379	.8418	1.2924
	44	.7578	.6980	.8438	.9355	1.4167
	45	.8047	.8005	.8848	.9531	1.3577
separated support	43	.7578	.7246	.8477	.9824	1.4646
	44	.8125	.7900	.9023	1.1035	1.4922
	45	.8203	.7949	.8965	1.0547	1.4896

Table 6: The three paired terminal MathIR rows. All runs end at step 4,608, corresponding to 12 passes.

H COMPONENT NECESSITY ABLATIONS

Main-body link. This appendix explains why each component of Figure 3 appears in Section 4, and how the terminal ablation in Section 7 fits the development record. A “remove-one” claim below means exactly what its evidence column permits: a mathematical necessity, a prospectively frozen developmental repair, an engineering isolation check, or a terminal causal comparison.

Fix removed	What breaks without it	Remove-one evidence and allowed conclusion
Executable gate and canonical key	Strings overcount formatting aliases, final answers erase distinct procedures, and invalid novelty can receive pressure.	Validator contracts and identity audits (Sections 3, Appendices B–D); definitional and engineering evidence, not a policy-effect estimate.
Predictive-bank centering	Empirical group-centered surprise is identically zero when every successful row has the same key, exactly where a diversity carrier is needed.	Lemma A.4 plus the zero-dispersion identity; analytic necessity. The one-time discovery term is not independently performance-isolated.
Verified-first cold start	A pre-verification token-entropy objective can increase uncertainty, length, or malformed outputs before any semantic object exists.	A prospectively frozen revision removed direct token entropy after its Python cold-start failure; developmental evidence, not a factorial comparison.
Verified-mass replay	Balance over retained keys is invariant to a common score shift and cannot by itself restore total probability on verified responses.	The score-gradient identity and the frozen replay-scaling repair sequence; analytic plus prospective developmental evidence.
Global replay scheduling	Prompt-local replay can leave a late verified discovery idle for nearly a full data pass.	One frozen revision changed only replay scheduling under an otherwise fixed protocol; prospective developmental evidence for the retained global scheduler, not a pooled historical effect.
Known-key balance replay	Uniform mass likelihood raises retained scores together but does not undo a monopoly among already covered correct modes.	Theorem A.5; analytic necessity conditional on coverage, not an isolated neural ablation.
Projection-free self-reference	Fixed or clipped pressure makes the bound, rather than the run’s own sensor, determine behavior and can silently saturate during collapse.	A prospectively frozen sequence of controller and carrier repairs; developmental evidence retained to explain the final controller, not pooled into Table 2.
Singleton support escape	Replay can preserve only observed keys; with one key there is no alternative carrier to rebalance.	Actuator off versus separated-support on: completed same-plumbing MathIR ablation, +0.1367 distinct@8 and +0.0267 pass@8 (Table 6). This is the sole terminal causal performance removal.
Separation of proposal support from on-policy counts	A proposal inserted into the count state changes the next on-policy novelty advantage, so the intervention is no longer support-only.	An earlier revision failed closed before optimization; the retained design splits $\mathcal{R}_x$ from $\mathcal{T}_x$. This proves isolation necessity, not performance gain.

Table 7: **Component-necessity ablations.** The evidence type is part of the result. Only the separated-support actuator row is a completed causal performance contrast; developmental rows explain why a fix exists without pretending that the sequence is a randomized factorial study.

Every developmental revision cited above is an immutable frozen record: failures, replacements, and exclusions are retained rather than rewritten, and no checkpoint or outcome is pooled across them into Table 2.

I DECODING CONTROL: DOES A WIDER SAMPLER REPAIR THE COLLAPSE?

Main-body link. This appendix supports the decoding statement in Section 7 and narrows the baseline limitation in Section 9.

Every headline cell samples at temperature one. A reader may reasonably ask whether matched Dr.GRPO’s missing modes are merely improbable rather than absent: if a modestly hotter sampler recovered them, the finding would concern sharpening at a fixed decoding setting, not loss of support. That is a different scientific claim, so we test it directly.

Protocol. We re-measure all 50 frozen terminal checkpoints (five domains $\times$ two arms $\times$ seeds 43–47) at six sampling temperatures $T \in \{0.5, 0.7, 1.0, 1.3, 1.6, 2.0\}$, giving 300 measurement cells. Each cell loads one checkpoint and runs the *same* evaluation path used during training—the same validator, prompts, split, response budget, $K = 8$, and four fixed draw seeds—while performing no rollout, optimizer step, or export, so a cell is a pure function of the checkpoint and its decoding setting. Two controls make the sweep admissible. First, every cell at $T = 1$ must reproduce its run’s published terminal value within $\max(3 \text{ SE}, .02)$ for rates and $\max(3 \text{ SE}, .05)$ for mode counts; all 200 such checks pass, the great majority exactly. Second, each cell runs on the GPU model that trained its checkpoint, because floating-point reduction order differs between GPU models and changes which tokens are sampled from identical weights under identical draw seeds. That effect is visible in the frozen pass-0 rows, which are identical within each GPU group and differ between them. It also bears on the headline cohort: twenty-two of the twenty-five paired seeds ran both arms on one GPU model and three pairs are split, so the sweep below pins placement where the headline design does not.

Estimand. For arm a and domain d define the *temperature-repair index*

$$\rho_{a,d} = \frac{\max_T \text{distinct@8}(a, d, T)}{\text{distinct@8}(\text{xGRPO}, d, 1.0)}.$$

The numerator maximizes over the whole sweep *with no accuracy constraint*, so the baseline may spend arbitrary correctness to buy breadth. A value below one means that no decoding setting recovers the treatment’s temperature-one breadth.

Domain	matched Dr.GRPO			xGRPO	
	best #modes	at T	m@8 there	#modes at $T=1$	ρ
Graph coloring	.327	1.3	.322	2.406	.14
Countdown	.650	1.6	.523	1.893	.34
Python factors	.172	0.5	.172	1.594	.11
MathIR action menu	.675	2.0	.363	.926	.73
PantryPlan	.719	2.0	.522	2.186	.33

Table 8: **Temperature does not repair the baseline.** Each row reports the largest `distinct@8` matched Dr.GRPO reaches at *any* of the six temperatures, the temperature that achieves it, and the accuracy it costs, against xGRPO at the reported temperature-one operating point. Every entry is a five-seed mean at the terminal pass 12 checkpoint.

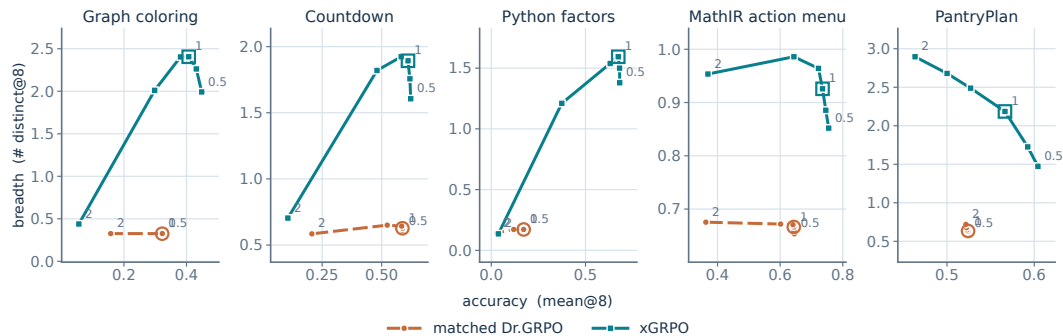


Figure 5: **Breadth against accuracy, parameterized by decoding temperature.** Each curve sweeps one arm’s terminal checkpoints over $T \in \{0.5, \dots, 2.0\}$; ringed markers are the reported temperature-one operating point and labels give T . Raising temperature moves a policy *along* its own curve, so “sample hotter” is a visible movement rather than an untested objection. The baseline curves are nearly vertical: accuracy falls while breadth stays where it was.

Result. In four of the five domains the baseline’s breadth is nearly invariant to temperature. Graph coloring moves $.323 \rightarrow .327$ across the entire range, and Python factors is constant at $.172$ through $T = 1.3$, while accuracy degrades sharply—Python mean@8 falls $.172 \rightarrow .029$ at $T = 2$. Heating the sampler produces invalid text, not a second verified execution. PantryPlan is the one domain where temperature does buy the baseline breadth at no accuracy cost ($.597 \rightarrow .719$ as T rises to 2 with mean@8 steady near $.52$), which its eight-token support-mask interface makes possible because it has no malformed-output failure mode; even there the gain closes about a third of the gap. Across all five domains $\rho \leq .73$, and in four of five xGRPO’s breadth at *every* temperature exceeds the baseline’s breadth at its *own best* temperature; on Python that holds at every $T < 2$, where both arms stop returning valid output.

The same sweep shows the reported operating point is not tuned in the treatment’s favor: xGRPO’s breadth peaks at $T = 1.0$ – 1.3 in four domains, so temperature one is at or near its own optimum. PantryPlan is again the exception, with xGRPO continuing to 2.896 at $T = 2$; reporting temperature one understates the treatment there.

We therefore read the failure as loss of support rather than sharpening: the absent modes are not waiting at low probability for a hotter sampler. This control covers decoding of the trained policies only. It says nothing about the alternative training objectives listed in Section 9.

J CLEAN COHORT PROGRESS AND CAUSAL PROVENANCE

Main-body link. This appendix provides the frozen terminal-cohort provenance behind Figure 6 and the evidence policy in Section 6.

1404
1405
1406
1407
1408
1409
1410
1411
1412
1413
1414
1415
1416
1417
1418
1419
1420
1421
1422
1423
1424
1425
1426
1427
1428
1429
1430
1431
1432
1433
1434
1435
1436
1437
1438
1439
1440
1441
1442
1443
1444
1445
1446
1447
1448
1449
1450
1451
1452
1453
1454
1455
1456
1457

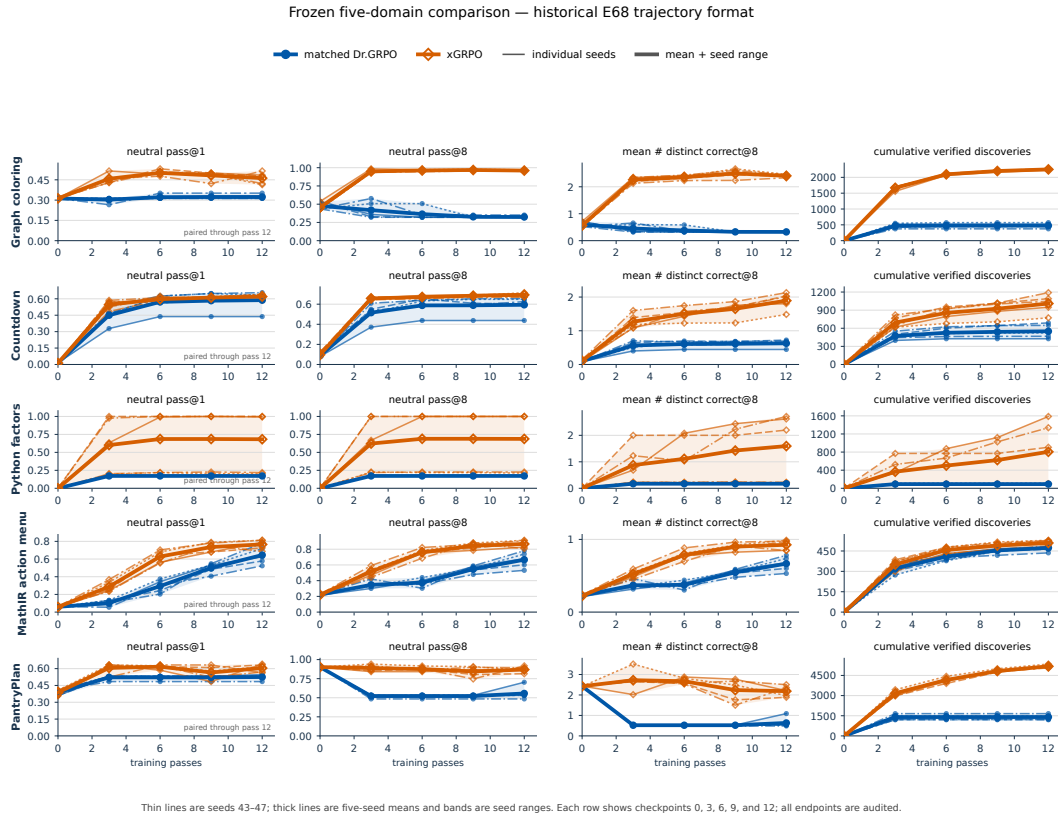


Figure 6: **Full five-domain, four-metric training grid.** Every row shows seeds 43–47 in both arms through the common audited pass-12 endpoint, on the one design of Section 6. The discovery column checks whether xGRPO continues accumulating verified keys after the matched baseline stalls.

1458
1459
1460
1461
1462
1463
1464
1465
1466
1467
1468
1469
1470
1471
1472
1473
1474
1475
1476
1477
1478
1479
1480
1481
1482
1483
1484
1485
1486
1487
1488
1489
1490
1491
1492
1493
1494
1495
1496
1497
1498
1499
1500
1501
1502
1503
1504
1505
1506
1507
1508
1509
1510
1511

Clean eight-environment 0.5B replacement campaign — historical E68 format
Four frozen outcome/discovery panels; 5 seeds × 12 passes; PointMaze geometry shift is a labeled replacement configuration
Stage A: 40/40 terminal, 40 metric-bearing, 40 materialized; 80/80 audited terminal overall
80/80 paper jobs submitted; remaining rows stay fail-closed; no historical values or carry-forward (2026-07-31 01:43 UTC)

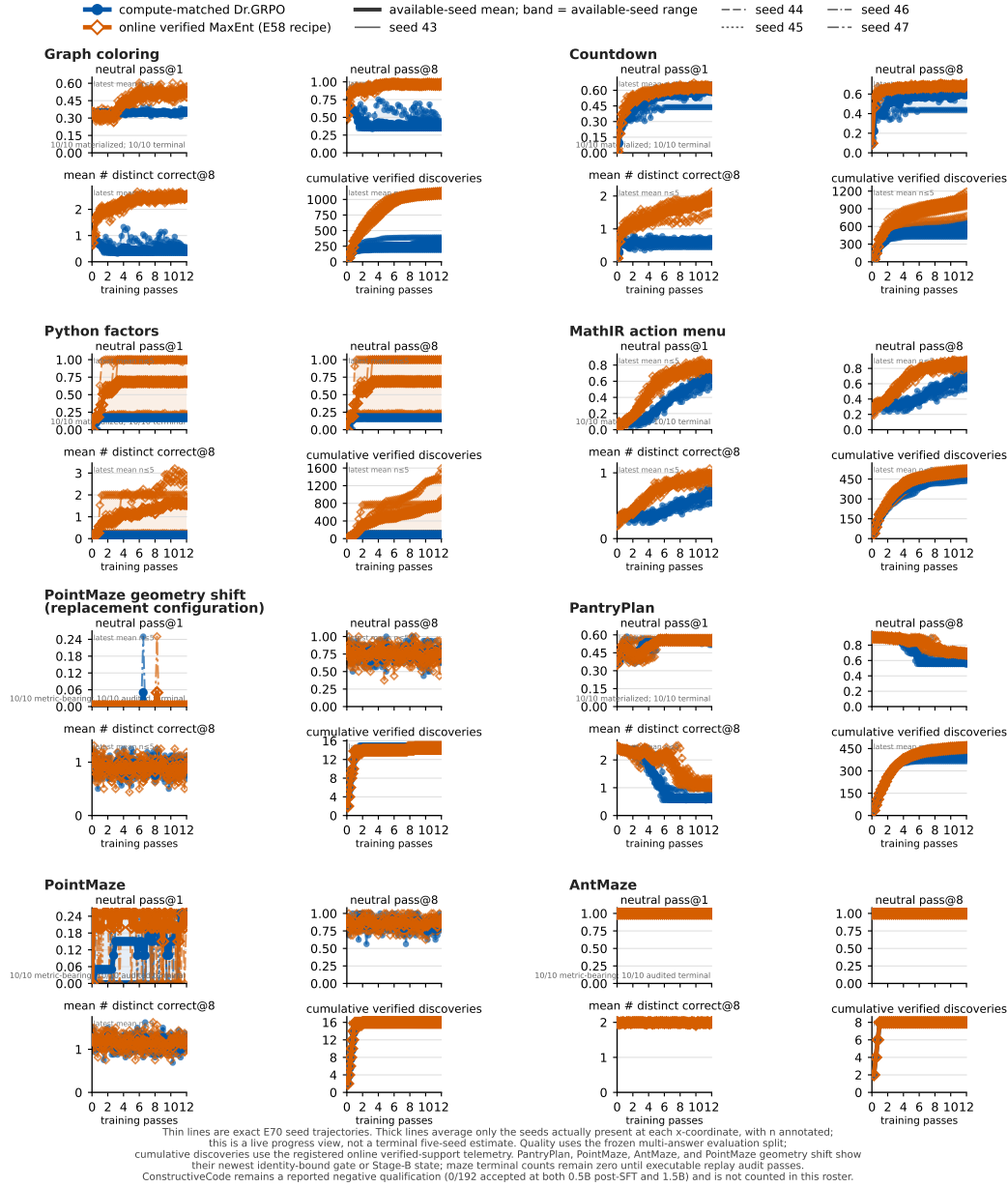


Figure 7: **Registered MATH-free eight-environment 0.5B comparison surface.** Each domain is a 2×2 card containing only neutral pass@1, neutral pass@8, mean # distinct correct@8, and cumulative verified discoveries. The five reported domains show identity-bound seed trajectories and means for the two frozen arms; the remaining cards show environments still in prospective replacement or admission states and are not reported in the paper’s surface. Incomplete means are labeled by contributing seed count; no missing seed is carried forward and no superseded result is copied into this surface.

K REPRODUCIBILITY AND CLAIM BOUNDARIES

Main-body link. This appendix enforces the evidence boundaries stated in Section 6 and Section 9.

Remaining hyperparameters. Every reported run uses learning rate 2×10^{-7} , one PPO epoch, top- $p = 1$, response length 192, and up to 4,608 updates. Treatment uses semantic coefficient .10, novelty credit $\nu = .50$, verified-mass and balance coefficients .10, EMA .9, 64-step warmups, surprisal clip $S = 5$, pseudocount $\lambda = 1$, and replay capacity $M = 16$. PantryPlan keeps those doses, optimizer settings, and five-seed design, but uses an eight-token response and a six-decision support mask.

Evidence and audit ledger.

Question	Comparison	Status	Allowed claim
Does xGRPO oppose collapse across the benchmark domains?	Graph, Countdown, Python, and MathIR at 384/128, 2 arms, 5 seeds, terminal pass 12	terminal	paired per-domain direction and magnitude on the common reporting rule
Does xGRPO retain PantryPlan support?	PantryPlan at 384/128, 2 arms, seeds 43–47, terminal pass 12	terminal	paired domain result on the common reporting rule
Does isolated support discovery act causally?	MathIR, same-plumbing actuator off/on, 3 seeds, pass 12	terminal	domain-specific mechanism result
Does wider decoding repair the baseline’s collapse?	All 50 terminal checkpoints re-measured at six temperatures, $K = 8$, 4 fixed draws, 300 cells	terminal	decoding-level control on frozen endpoints; no claim about alternative training objectives

Table 9: **Evidence hierarchy.** Scope is determined by completion and prospective controls, not by whether a result is favorable.

Bank state, controller state, scheduler cursor, optimizer state, data cursor, and request-stream state are checkpointed together. Non-finite values, state mismatch, algorithm exceptions, missing terminal markers, proposal leakage into PPO, or proposal overlap with on-policy counts are fail-closed violations. Placement-only recovery amendments are hash-bound. Two superseded cohorts are excluded by machine-readable objective audits rather than by their outcomes. Figure 1 is generated by `ops/plot_paper_collapse_toy.py` directly from the retained seed-43 Qwen2.5-0.5B-Instruct sample logs. Its prompt-selection rule, displayed counts, source paths, and source hashes are frozen in `var/artifacts/paper_graph_collapse_toy.json`. Figure 2 is generated by `ops/plot_paper_modebench_examples.py`; its five depicted instance IDs are bound to the released evaluation splits under `var/data/`. The renderer rechecks every displayed coloring edge, factor, arithmetic result, and algebra state transition before writing the figure. Figure 3 is the component schematic generated by `ops/plot_paper_xdr_mechanism.py`; it contains no empirical values. Table 2 and Figure 6 read `var/artifacts/gce71_scale384_05b_12pass_scaling_curve.json`, `var/artifacts/cde70_clean_stage_a_05b_12pass_scaling_curve.json`, `var/artifacts/pye70_clean_stage_a_05b_12pass_scaling_curve.json`, `var/artifacts/mie70_clean_stage_a_05b_12pass_scaling_curve.json`, and `var/artifacts/ppe71_scale384_05b_12pass_scaling_curve.json`. Rendering fails unless `var/artifacts/e70_clean_stage_a_05b_audit_latest.json` records all 40 run states as audited terminal and `var/artifacts/pantry_stage_b_05b_12pass_audit.json` records the ten-cell PantryPlan terminal decision for seeds 43–47. Table 6 is reproduced from `var/artifacts/e66_e68_mathir_terminal_recovery_summary.json`. Table 8 and Figure 5 are produced by `ops/exp_scaling/build_e72_frontier_manifest.py`, `ops/exp_scaling/launch_e72_decoding_frontier.py`, `ops/exp_scaling/aggregate_e72_frontier.py`, and `ops/exp_scaling/plot_e72_decoding_frontier.py` from `var/artifacts/e72_decoding_frontier_summary.json`, with one row per measured cell in `var/artifacts/e72_decoding_frontier_cells.jsonl` and the checkpoint inventory in `var/artifacts/e72_frontier_source_runs.json`.

Each cell inherits its domain's evaluation configuration from the source run's own recorded arguments, and rendering aborts unless every temperature-one cell reproduces its published terminal value. The eight-row progress surface is rendered in the four-panel format by `ops/exp_scaling/plot_e70_clean_05b_wide_live.py`, with hashes in `var/artifacts/clean_05b_eight_environment_figure_provenance.json`. The headline protocols are `paper/preregistration/e70_clean_verified_maxent_vs_compute_matched_drgrpо_stage_a_05b.md` and `paper/preregistration/pantry_stage_b_05b_12pass_20260730.md`; the superseded earlier protocol remains `paper/preregistration/e61r1_e58_vs_grpo_05b_12pass.md`. The mature base objective and same-plumbing ablations are bound by `paper/preregistration/e58_global_verified_replay_canonical_05b.md`, `paper/preregistration/e66_same_plumbing_actuator_ablation_05b.md`, and `paper/preregistration/e68_separated_support_actuator_ablation_05b.md`.

The environments are implemented in `src/oat_drgrpо/canonical_actions.py`, `src/oat_drgrpо/python_modebench.py`, `src/oat_drgrpо/mathir.py`, and `src/oat_drgrpо/pantry_plan.py`. The external Python process boundary is in `src/oat_drgrpо/python_modebench_process.py` and its worker in `src/oat_drgrpо/python_modebench_worker.py`. The open-set signal is in `src/oat_drgrpо/semantic_shannon.py`, replay in `src/oat_drgrpо/canonical_replay.py`, the bank in `src/oat_drgrpо/online_canonical_bank.py`, and the separated actuator in `src/oat_drgrpо/counterfactual_proposals.py`.

The manuscript separates four evidence classes:

1. exploratory design evidence, used to motivate components but never pooled into the final method comparison;
2. the completed five-seed, five-domain surface at pass 12, reported one domain at a time rather than as a single pooled effect;
3. the terminal six-run MathIR causal ablation; and
4. superseded, repaired, or failed cohorts, retained in the repository but excluded from every headline estimate.